

Understanding APIs & JSON

A hands-on walkthrough using Postman and the Simple Books API

Quick Recap: The Menu Analogy

Earlier, we compared calling an API to ordering from a menu instead of cooking the dish yourself — you don’t need to know how the kitchen works, you just need to know what to ask for.

That’s a useful way to *feel* what an API does. Now let’s open the hood and look at what’s actually happening — the real mechanics behind that analogy, using a real, free practice API and Postman.

What Is an API, Really?

An API (Application Programming Interface) is a defined way for one piece of software to ask another piece of software to do something, over the internet, using a standard set of rules. Concretely, it involves:

- **A client** — the thing making the request (Postman, a Python script, a mobile app)
- **A server** — the thing that receives the request, does something, and sends back a result
- **An endpoint** — the specific web address the request is sent to, like `https://simple-books-api.glitch.me/books`
- **An HTTP method** — a word that tells the server *what kind of action* you want:

Method	What it means	Everyday equivalent
GET	Fetch/read information	Looking something up
POST	Create something new	Submitting a form
PATCH	Update part of something that exists	Editing one field
DELETE	Remove something	Deleting a record

- **Headers** — extra information sent alongside the request, such as `Content-Type` (what format the data is in) or `Authorization` (proof of who you are)
- **A status code** — a short number the server sends back telling you what happened:

Code	Meaning
200	OK — the request succeeded
201	Created — something new was successfully created
400	Bad Request — something about your request was malformed
401	Unauthorized — you’re missing or have an invalid access token
404	Not Found — the thing you asked for doesn’t exist
409	Conflict — the thing you’re trying to create already exists

Put together: an API call is a client sending an HTTP method to an endpoint, sometimes with headers and data attached, and getting back a status code plus a result.

What Is JSON, Really?

JSON (JavaScript Object Notation) is the text format almost every API uses to send and receive data. It organizes information into simple, readable key-value pairs — once you see the structure, it’s easy to read even before writing any code yourself. (When we get to Python later in the course, you’ll notice JSON looks identical to Python’s own dictionaries and lists — but you don’t need Python to understand it here.)

The rules are simple:

- Data is organized into **key-value pairs**: `"key": value`
- An **object** (a single record) is wrapped in curly braces `{ }`
- A **list of items** is wrapped in square brackets `[ ]`
- Text values are wrapped in double quotes; numbers and `true / false` are not

```
{
  "id": 1,
  "name": "The Russian",
  "type": "fiction",
  "available": true
}
```

Objects can be nested inside each other, and lists can hold several objects at once — this is exactly the shape you'll see in every response from the Simple Books API below:

```
[
  { "id": 1, "name": "The Russian", "type": "fiction" },
  { "id": 2, "name": "Just as I Am", "type": "non-fiction" }
]
```

Meet the Simple Books API

For the rest of this guide, we'll use the **Simple Books API** — a free, public practice API built specifically for learning how to make real API calls. It isn't connected to a real bookstore; it exists purely so students can practice safely.

Base URL: `https://simple-books-api.glitch.me`

We'll walk through it endpoint by endpoint, in Postman, in the order shown below.

01 — Hello Book API

GET /

The simplest possible request — just to confirm Postman can reach the API at all and see what a basic JSON response looks like.

02 — Status

GET /status

Checks whether the API is up and running. A quick way to sanity-check your connection before doing anything else.

```
Response:
{ "status": "OK" }
```

03 — List of Books

GET /books

Returns every book in the system as a JSON array. This endpoint also accepts optional query parameters to narrow the results:

- `?type=fiction` or `?type=non-fiction` — filter by category
- `?limit=5` — limit how many results come back

Example: `GET /books?type=fiction&limit=2`

```
Response:
[
  { "id": 1, "name": "The Russian", "type": "fiction", "available": true },
  { "id": 4, "name": "The Great Gatsby", "type": "fiction", "available": true }
]
```

04 — Get a Single Book

`GET /books/{id}`

Notice the `{id}` in the URL — this is called a **path parameter**. Instead of a fixed address, part of the URL changes depending on which book you want. `GET /books/1` asks for the book with ID 1 specifically.

```
Response:
{ "id": 1, "name": "The Russian", "type": "fiction", "available": true }
```

If you ask for a book ID that doesn't exist, the API responds with a **404** status code instead.

05 — API Authentication

`POST /api-clients`

Everything so far has been open to anyone. But submitting, viewing, updating, or deleting an *order* requires proving who you are first — that's what the  icon means on those endpoints in the Postman collection.

To get an access token, you register as an API client by sending your name and email:

```
Request body:
{
  "clientName": "Ayesha",
  "clientEmail": "ayesha@example.com"
}
```

```
Response:
{
  "accessToken": "3f9a1c...",
  "expires": "7 days"
}
```

Copy that `accessToken` — every locked endpoint from here on needs it included as a header:

```
Authorization: Bearer 3f9a1c...
```

If you try to reuse the same email to register twice, the API responds with a **409 Conflict**, since that client is already registered.

06 — Submit an Order

`POST /orders`

Now that we have a token, we can create something. This request needs the `Authorization` header from Step 05, plus a JSON body describing what to order:

```
Headers:
Authorization: Bearer 3f9a1c...

Request body:
{
  "bookId": 1,
  "customerName": "Ayesha"
}
```

A successful request returns status **201 Created**, along with the new order's ID.

07 — Get All Orders

`GET /orders`

Returns a JSON array of every order associated with your access token. Requires the `Authorization` header — try it without one first, and notice the **401 Unauthorized** response.

08 — Get an Order

GET /orders/{id}

Same path-parameter pattern as fetching a single book, but for one specific order.

09 — Update an Order 🔒

PATCH /orders/{id}

Changes *part* of an existing order without needing to resend the whole thing — for example, updating just the customer name:

```
Request body:
{ "customerName": "Ayesha Khan" }
```

This is the difference between PATCH and a full replace: you only send the field that’s changing.

10 — Delete an Order 🔒

DELETE /orders/{id}

Removes the order entirely. No request body needed — the endpoint in the URL is enough to identify what to delete. A successful delete returns status **200** or **204**, and a follow-up GET on the same order ID now returns a **404**.

Why This Order Matters

Notice the sequence isn’t random: read-only, unlocked endpoints first (01-04), then authentication (05), then the locked create/read/update/delete cycle on orders (06-10). This mirrors almost every real API you’ll ever use — including Huawei Cloud EI. Once you’re comfortable with this pattern here, calling a Huawei EI endpoint later in the course will feel familiar rather than brand new.

Quick Quiz

1. What does the {id} inside a URL like /books/{id} represent?
2. Which HTTP method would you use to change just one field of an existing order, without resending the whole thing?
3. What status code tells you your request was missing a valid access token?
4. Why does registering an API client with an email that’s already registered return a 409 instead of a 201?
5. In JSON, what’s the difference between { } and []?

Glossary Additions

Term	Meaning in plain words
Client	The thing making an API request (Postman, an app, a script)
Endpoint	The specific web address an API request is sent to
HTTP method	The action word in a request — GET, POST, PATCH, DELETE
Path parameter	A variable part of a URL, like the {id} in /books/{id}
Query parameter	Extra options added to the end of a URL, like ?type=fiction
Header	Extra information sent with a request, separate from the main body
Status code	A number the server sends back describing what happened
Access token	A string proving who you are, required for locked/protected endpoints
JSON	The text format almost every API uses to represent structured data

Answer Key

1. A **path parameter** — a placeholder in the URL that gets replaced with a specific value, like a book or order ID.
2. **PATCH** — it updates part of an existing resource instead of replacing the whole thing.
3. **401 Unauthorized**.
4. Because that email is already tied to an existing API client — the server is telling you the resource you're trying to create already exists, which is exactly what a 409 Conflict means.
5. `{ }` defines a single **object** (key-value pairs describing one thing); `[ ]` defines a **list** of items, which can itself contain multiple objects.